



**T.C.**

**SAKARYA ÜNİVERSİTESİ**

**BİLGİSAYAR VE BİLİŞİM BİLİMLERİ FAKÜLTESİ**

**BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**

**SİSTEM PROGRAMLAMA**

**SİSTEM PROGRAMLAMA DERSİ KISA SINAV ÖDEVLERİ**

**B221210382-Emir Furkan ASLAN**

**B221210400-Melih ERÖZ**

**GitHub Adresi: <https://github.com/furkanleone/sistemodev>**

**Dr. Öğr. Üyesi Hüseyin ESKİ**

**SAKARYA**

**20 Mayıs ,2026**

## 1. Giris

Bu rapor, Sistem Programlama dersi kısa sınav ödevi kapsamında verilen iki programlama probleminin çözümünü açıklamaktadır. Birinci problemde dinamik olarak ayrılmış tek boyutlu bellek bloğu üzerinde işaretçi aritmetiğiyle matris köşegen toplamı hesaplanmıştır. İkinci problemde ise fork ile oluşturulan çocuk süreç üzerinde alarm ve sinyal çağrıları kullanılarak süreç durdurma, devam ettirme ve sonlandırma davranışı gösterilmiştir.

## 2. Soru 1: Dinamik Matris ve İşaretçi Aritmetiği

### 2.1 Problem Tanımı

Bu soruda  $N \times N$  boyutunda bir matrisin malloc ile dinamik olarak ayrılması ve matris elemanlarına yalnızca işaretçi aritmetiği kullanılarak erişilmesi istenmiştir. Program, ana diyagonal ve ikincil köşegen elemanlarının toplamını hesaplamalı; matris boyutu tek sayı olduğunda merkez elemanı iki kez toplamamalıdır.

### 2.2 Çözüm Mantığı

- Matris bellekte tek boyutlu int işaretçisiyle,  $N*N$  elemanlık bir blok olarak tutulmuştur.
- Her elemanın adresi  $mat + i * cols + j$  formülüyle hesaplanmıştır.
- Ana köşegen için  $i == j$  konumundaki değerler toplanmıştır.
- İkincil köşegen için  $j == cols - 1 - i$  konumundaki değerler toplanmıştır.
- Merkez eleman ana ve ikincil köşegenin kesiştiği durumda yalnızca bir kez eklenmiştir.
- Program sonunda ayrılan bellek free ile serbest bırakılmıştır.

### 2.3 Kaynak Kod Açıklaması

special\_sum fonksiyonu, matris üzerinde satır satır dolaşarak köşegen elemanlarını hesaplar. Fonksiyon içinde iki boyutlu dizi indeksleme sözdizimi kullanılmaz; tüm erişimler işaretçi aritmetiği ile yapılır. main fonksiyonu kullanıcıdan matris boyutunu ve elemanları alır, belleği malloc ile ayırır, sonucu hesaplatır ve ayrılan belleği serbest bırakır.

### 2.4 soru1\_matris.c

```
/*
 * Soru 1: Dinamik 2D Matris - Yalnızca İşaretçi Aritmetiği Kullanarak
 * Derleme: gcc -o soru1_matris soru1_matris.c
 * Çalıştırma: ./soru1_matris
 */

#include <stdio.h>
#include <stdlib.h>

/*
 * special_sum: N x N matrisin ana köşegen ve ikincil köşegen
 * elemanlarının toplamını hesaplar. Yalnızca işaretçi aritmetiği kullanılır.
 * Merkez eleman (eger varsa) iki kez sayılmaz.
 *
 * Parametreler:
 *   mat - Matrisin başlangıç adresini gösteren işaretçi (1D bellek bloğu)
 *   rows - Matrisin satır sayısı
 *   cols - Matrisin sütun sayısı
 *
 * Donus: Köşegen elemanlarının toplamı (int)
 */
int special_sum(int *mat, int rows, int cols) {
    int sum = 0;
    int i;
```

```

for (i = 0; i < rows; i++) {
    /* Ana kosegen elemani: pozisyon (i, i)
    * Bellekteki karsilik: mat + i * cols + i
    * mat[i][j] sozdizimi KULLANILMIYOR */
    sum += *(mat + i * cols + i);

    /* Ikincil kosegen elemani: pozisyon (i, cols-1-i)
    * Bellekteki karsilik: mat + i * cols + (cols - 1 - i) */
    int secondary_col = cols - 1 - i;
    if (secondary_col != i) {
        /* Merkez eleman degil: iki kez saymamak icin kontrol */
        sum += *(mat + i * cols + secondary_col);
    }
    /* Eger secondary_col == i ise, bu eleman ana kosegen elemanidir
    * (merkez eleman), zaten yukarida eklendi, tekrar eklenmez. */
}

return sum;
}

int main(void) {
    int n;          /* Matris boyutu (NxN) */
    int *mat;       /* Matrisi temsil eden tek boyutlu isaretci */
    int i, j;       /* Dongu degiskenleri */
    int result;     /* special_sum sonucu */

    /* Kullanicidan matris boyutunu al */
    printf("Matris boyutunu girin (N): ");
    if (scanf("%d", &n) != 1 || n <= 0) {
        fprintf(stderr, "Gecersiz boyut!\n");
        return 1;
    }

    /* malloc ile N*N boyutunda bellek tahsis et (tek 1D blok) */
    mat = (int *)malloc(n * n * sizeof(int));
    if (mat == NULL) {
        fprintf(stderr, "Bellek tahsisi basarisiz!\n");
        return 1;
    }

    /* Kullanicidan matris elemanlarini oku
    * Eleman (i,j) bellekte: mat + i*n + j konumundadir */
    printf("Matris elemanlarini satir satir girin (%d x %d):\n", n, n);
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            printf("mat[%d][%d] = ", i, j);
            /* Isaretci aritmetigi ile elemana eris */
            if (scanf("%d", (mat + i * n + j)) != 1) {
                fprintf(stderr, "Gecersiz giris!\n");
                free(mat);
                return 1;
            }
        }
    }

    /* Girilen matrisi ekrana yazdir (dogrulama amaclli) */
    printf("\nGirilen Matris:\n");
    for (i = 0; i < n; i++) {
        for (j = 0; j < n; j++) {
            /* Isaretci aritmetigi ile okuma: *(mat + i*n + j) */
            printf("%5d", *(mat + i * n + j));
        }
        printf("\n");
    }

    /* special_sum fonksiyonunu cagir ve sonucu yazdir */
    result = special_sum(mat, n, n);
    printf("\nAna + Ikincil Kosegen Elemanlari Toplami = %d\n", result);

    /* Tahsis edilen bellek serbest birakiliyor - bellek sizintisi onlenir */
    free(mat);
}

```

```
mat = NULL;  
  
return 0;  
}
```

## 2.5 Çalışma Çıktısı

Soru 1 programının Ubuntu/Linux ortamında çalıştırılması sonucunda 3x3 matris için toplam değer 25 olarak elde edilmiştir.

Şekil 1. Soru 1 çalışma çıktısı

```
b2212@10400:~/b221210382_b221210400$ ./soru1_matris  
Matris boyutunu girin (N): 3  
Matris elemanlarını satır satır girin (3 x 3):  
mat[0][0] = 1  
mat[0][1] = 2  
mat[0][2] = 3  
mat[1][0] = 4  
mat[1][1] = 5  
mat[1][2] = 6  
mat[2][0] = 7  
mat[2][1] = 8  
mat[2][2] = 9  
  
Girilen Matris:  
  1   2   3  
  4   5   6  
  7   8   9  
  
Ana + İkinci Kosegen Elemanları Toplamı = 25
```

## 3. Soru 2: Sinyal İşleme ve Süreç Kontrolü

### 3.1 Problem Tanımı

Bu soruda SIGALRM, SIGINT, SIGSTOP ve SIGCONT sinyalleri kullanılarak süreçler arası sinyal işleme davranışının gösterilmesi istenmiştir. Program fork ile bir çocuk süreç oluşturmali, çocuk süreç sayaç yazdırmalı, ana süreç ise alarm mekanizmasıyla çocuğu belirli aralıklarla durdurup devam ettirmelidir.

### 3.2 Çözüm Mantığı

- Program başlangıcında fork çağrısıyla çocuk süreç oluşturulmuştur.
- Çocuk süreç sonsuz döngü içinde her saniye sayaç değerini yazdırmaktadır.
- Çocuk süreç SIGINT sinyalini yakaladığında programı sonlandırmadan bilgilendirme mesajı yazdırır.
- Çocuk süreç SIGCONT sinyalini yakaladığında işlemin devam ettiğini belirten mesajı yazdırır.

- SIGSTOP işlenemeyen bir sinyal olduğu için handler tanımlanmamış, yalnızca ana süreç tarafından gönderilmiştir.
- Ana süreç alarm(3) ile 3 saniyelik tetikleme başlatır; alarm geldiğinde çocuğa SIGSTOP gönderir, 2 saniye bekler ve SIGCONT gönderir.
- İkinci durdurma/devam ettirme döngüsünden sonra ana süreç çocuğa SIGINT gönderir, ardından çocuk süreci sonlandırır ve programdan çıkar.

### 3.3 Sinyal ve Süreç Akışı

Ana süreç SIGALRM yakalayıcısı içinde çocuk sürecin PID değerini kullanarak kill fonksiyonuyla sinyal gönderir. İlk aşamada çocuk SIGSTOP ile durdurulur. İki saniyelik beklemeden sonra SIGCONT gönderilerek çocuk süreç devam ettirilir ve SIGCONT handler mesajı yazdırılır. İkinci devam ettirme sonrasında SIGINT hemen gönderilerek fazladan sayaç çıktıları azaltılır; ardından süreç sonlandırılır.

### 3.4 Kaynak Kod Açıklaması

child\_sigint\_handler fonksiyonu SIGINT alındığında bilgilendirme mesajı yazdırır. child\_sigcont\_handler fonksiyonu SIGCONT sonrasında işlemin devam ettiğini bildirir. parent\_sigalrm\_handler fonksiyonu alarm tetiklendiğinde çocuk sürece SIGSTOP ve SIGCONT göndererek süreç kontrolünü gerçekleştirir. Program, POSIX/Linux sistem çağrılarını kullandığından Ubuntu gibi Linux ortamında derlenip çalıştırılmalıdır.

### 3.5 soru2\_sinyal.c

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <time.h> /* nanosleep() için eklendi */

volatile pid_t child_pid = 0;
volatile int alarm_count = 0;

/* Çocuk surec SIGINT aldığında programı hemen bitirmez;
 * istenen bilgilendirme mesajını yazdırıp çalışmaya devam eder. */
void child_sigint_handler(int sig) {
    (void)sig; /* Kullanılmayan parametre uyarisi susturuldu */
    signal(SIGINT, child_sigint_handler);
    printf("Çocuk: SIGINT alındı ancak devam ediliyor...\n");
    fflush(stdout);
}

/* Çocuk surec SIGCONT ile devam ettirildiğinde bu handler çalışır
 * ve surecin tekrar çalışmaya başladığını bildirir. */
void child_sigcont_handler(int sig) {
    (void)sig;
    signal(SIGCONT, child_sigcont_handler);
    printf("Çocuk: İşlem devam etti\n");
    fflush(stdout);
}

/* Ana surecin alarm handler'i her tetiklemede çocuk sureci önce
 * SIGSTOP ile durdurur, 2 saniye bekler ve SIGCONT ile devam ettirir.
 * İkinci alarmdan sonra SIGINT gönderip çocuk sureci sonlandırır. */
void parent_sigalrm_handler(int sig) {
    (void)sig;
    signal(SIGALRM, parent_sigalrm_handler);

    alarm_count++;
```

```

if (child_pid <= 0) return;

printf("Ebeveyn: Çocuk durduruluyor...\n");
fflush(stdout);
/* SIGSTOP islenemez; bu nedenle çocuk sureci dogrudan durdurur. */
kill(child_pid, SIGSTOP);

/* Odev isterine uygun olarak çocuk surec 2 saniye durdurulmus kalir. */
sleep(2);

if (alarm_count < 2) {
    printf("Ebeveyn: Çocuk devam ediyor...\n");
    fflush(stdout);
    /* Çocuk surecin calismasina devam etmesi icin SIGCONT gonderilir. */
    kill(child_pid, SIGCONT);
    alarm(3);
} else {
    printf("Ebeveyn: Çocuk devam ediyor...\n");
    fflush(stdout);
    /* Son dongude de SIGCONT gonderilir, ardindan bitirme asamasina gecilir. */
    kill(child_pid, SIGCONT);

    printf("Ebeveyn: SIGINT gönderiliyor...\n");
    fflush(stdout);
    /* Çocuk surecin SIGINT handler'inin calistigini gostermek icin SIGINT gonderilir. */
    kill(child_pid, SIGINT);

    /* POSIX-2008 uyumlu 0.1 saniyelik bekleme (usleep yerine) */
    struct timespec req = {0, 100000000};
    nanosleep(&req, NULL);

    /* Çocuk surec temiz bicimde beklenmeden once kesin olarak sonlandirilir. */
    kill(child_pid, SIGKILL);
    waitpid(child_pid, NULL, 0);

    printf("Ebeveyn: Çocuk sonlandırıldı.\n");
    fflush(stdout);
    exit(0);
}
}

int main(void) {
    pid_t pid;
    int counter = 0;

    pid = fork();

    if (pid < 0) {
        perror("fork hatası");
        return 1;
    }

    if (pid == 0) {
        /* ÇOCUK SÜREÇ */
        /* Çocuk surec kendisine gelen SIGINT ve SIGCONT sinyallerini yakalar. */
        signal(SIGINT, child_sigint_handler);
        signal(SIGCONT, child_sigcont_handler);

        /* Çocuk surec sonsuz dongude her saniye sayac degerini yazdirir. */
        while (1) {
            printf("Çocuk sayacı: %d\n", counter);
            fflush(stdout);
            counter++;
            sleep(1);
        }
        return 0;
    }
}

```

```

/* ANA SÜREÇ */
/* Ana surec cocugun PID degerini saklar ve SIGALRM icin handler kurar. */
child_pid = pid;
signal(SIGALRM, parent_sigalrm_handler);

/* Ilk alarm 3 saniye sonra tetiklenir. Handler sonraki alarmi tekrar kurar. */
alarm(3);

while (1) {
    pause();
}

return 0;
}

```

### 3.6 Çalışma Çıktısı

Soru 2 programının Ubuntu/Linux ortamında çalıştırılması sonucunda çocuk sürecin sayaç yazdırdığı, ana sürecin çocuğu SIGSTOP ile durdurup SIGCONT ile devam ettirdiği, SIGINT sinyalinin yakalandığı ve çocuk sürecin sonlandırıldığı görülmüştür.

Şekil 2. Soru 2 çalışma çıktısı

```

b2212@10400:~/b221210382_b221210400$ ./soru2_sinyal
Çocuk sayacı: 0
Çocuk sayacı: 1
Çocuk sayacı: 2
Ebeveyn: Çocuk durduruluyor...
Ebeveyn: Çocuk devam ediyor...
Çocuk: İşlem devam etti
Çocuk sayacı: 3
Çocuk sayacı: 4
Çocuk sayacı: 5
Ebeveyn: Çocuk durduruluyor...
Ebeveyn: Çocuk devam ediyor...
Ebeveyn: SIGINT gönderiliyor...
Çocuk: İşlem devam etti
Çocuk: SIGINT alındı ancak devam ediliyor...
Çocuk sayacı: 6
Ebeveyn: Çocuk sonlandırıldı.

```

### 4. Sonuç

Hazırlanan iki C programı ödevde istenen temel sistem programlama kavramlarını karşılamaktadır. Birinci program dinamik bellek yönetimi ve işaretçi aritmetiğini, ikinci program ise süreç oluşturma ve sinyal gönderme/yakalama mekanizmalarını göstermektedir. Çalışma çıktısı ekran görüntüleri ilgili bölümlere eklendiğinde rapor teslim formatını tamamlayacaktır.